

AgentOS — Plan de trabajo pendiente

Documento de traspaso para otro LLM (DeepSeek 4.1-flash)

Fecha: 3 de octubre de 2026

Repositorio: <https://github.com/ccarrillomanzanares/newos> (rama `main`)

0. Cómo usar este documento

Este documento describe **todo lo que queda por hacer** para completar AgentOS, el objetivo original del usuario. Está escrito para que un LLM distinto pueda continuar el trabajo sin acceso a la conversación previa.

Reglas de lectura:

1. La sección 4 dice **qué está hecho y verificado** y qué no. No asumas que algo funciona solo porque el código exista.
 2. Las secciones 5 a 12 son los bloques de trabajo, **en orden de prioridad**. Cada uno tiene objetivo, ficheros, pasos y criterio de aceptación.
 3. La sección 13 lista **errores ya resueltos**. Léela antes de tocar el build: son trampas que ya costaron varias iteraciones.
 4. Todo el texto visible al usuario debe estar **en español**.
-

1. Objetivo original del proyecto

Palabras del usuario:

«Quiero crear un nuevo sistema operativo desde 0, que sea un agente conversacional que pueda administrarse a sí mismo con todo el conocimiento de TI posible y su LLM también lo quiero crear desde 0. La interfaz del SO ha de ser lo más sencilla posible, ha de permitir comunicarse por voz y también por texto y cuando sea necesario mostrar información al usuario, es decir, cualquier aplicación gráfica: editores, película, ssh, navegador... etc.»

De ahí se derivan **siete pilares**:

#	Pilar	Estado
1	SO propio (Linux + Buildroot, ISO híbrido BIOS/UEFI)	Casi completo
2	Agente conversacional que se autoadministra	Parcial
3	Conocimiento de TI (documentación del sistema accesible al agente)	No empezado
4	LLM propio (entrenado/ajustado desde cero)	No empezado
5	Interfaz mínima estilo Her (sin iconos, sin barras, orbe)	Hecho
6	Voz (STT + TTS)	Esbozado, sin integrar
7	Aplicaciones gráficas (navegador, editor, ssh, vídeo)	Parcial

Hardware objetivo: portátil x86_64 antiguo, 8 GB de RAM (≈6-7 GB efectivos, gráficos integrados comparten memoria).

2. Entorno y accesos

Máquina de compilación (VPS del usuario):

- Host: `ccmai@vmi3117361` , Ubuntu 24.04, 16 vCPU
- Proyecto: `~/newos` (clon del repositorio)
- Buildroot 2024.11.1, clonado en `build/buildroot` , salida en `build/output`

Comando de compilación (obligatorio tal cual):

```
cd ~/newos
git pull
HOSTCC=gcc-12 HOSTCXX=g++-12 make build J=16
```

Requiere `sudo apt install gcc-12 g++-12` (ver sección 13, trampa 2).

Salidas del build:

- `build/output/images/agentos.iso` — DVD/VM, sistema en RAM
- `build/output/images/agentos.img` — USB, sistema persistente

Credenciales por defecto del sistema instalado:

- Usuario `agent` , contraseña `agent`

- Usuario `root` , contraseña `agent`

Backend LLM: Ollama Cloud (<https://ollama.com/v1>). El usuario usa el modelo `deepseek-v4.1-flash` . La clave se guarda **fuera del repositorio**, en `build/overlays/etc/default/agentos` (está en `.gitignore`).

Rutas internas del sistema en ejecución:

- Código: `/opt/agentos`
- Socket de AgentD: `/run/agentos/input.sock`
- Interfaz web: puerto `8080`
- Configuración: `/etc/default/agentos`
- Marcador de configuración: `/etc/agentos/.configured`
- Logs: `/var/log/agentd.log` , `/var/log/agentos-gui.log`

3. Reglas de trabajo impuestas por el usuario

Estas reglas son **permanentes** y no se pueden relajar:

1. **Todo el texto visible al usuario, en español.**
2. **Confirmación obligatoria antes de comandos destructivos** (`rm -rf` , formateos, `wipefs`). El instalador ya pide escribir `BORRAR` para confirmar; mantén ese patrón.
3. **Nunca escribir claves API en el código ni en el repositorio.** Se referencian por nombre de variable (`OLLAMA_API_KEY`) o se leen de `/etc/default/agentos` .
4. **No reintroducir el `MockBackend`** — el usuario pidió eliminarlo.
5. **La voz es lo último** que se implementa (ya no aplica: es el siguiente bloque tras el arranque).
6. **El sistema debe arrancar solo.** El usuario solo debe introducir la API key o la ruta del modelo, y solo la primera vez.

4. Estado actual (verificado)

4.1 Hecho y funcionando

- **Buildroot compila** y genera `agentos.iso` y `agentos.img` híbridos (BIOS + UEFI, GRUB2).
- **Kernel Linux 6.6** con fragmento propio (`build/configs/kernel_config`), arranque silencioso (`loglevel=0` , sin logo, sin cursor).
- **AgentD:** servidor de socket UNIX (`core/ipc/socket_server.py`) con agente (`core/agent/agent.py`), gestión de contexto (`core/llm/context_manager.py`) e inferencia (`core/llm/inference.py`).
- **Interfaz Her:** servidor WebSocket (`ui/gui/server.py`), lanzador kiosk (`ui/gui/launch.py`) y frontend con orbe animado (`ui/gui/static/index.html`).
- **Chat de terminal** (`ui/tui/chat.py`) para desarrollo.
- **Instalador en disco** (`board/agentos/install-to-disk.sh` → comando `agentos-install`).
- **Asistente de primer arranque** (`board/agentos/first-boot.sh`) que pide proveedor LLM y API key o ruta del modelo.
- **Scripts de desarrollo:** `scripts/run_dev.sh` , `make gui` , `make dev-cloud` .
- **Tests:** `tests/test_core.py` (3 pasando).

4.2 Escrito pero NO verificado

- **Arranque automático hasta la interfaz.** Se añadieron `S49agentos-firstboot`, `S71weston`, `weston.ini`, `seatd` y el autologin por `inittab`. **Nada de esto se ha probado todavía en una imagen recién compilada.** La última captura del usuario muestra el comportamiento antiguo (pide login en `tty1`), lo que indica que esa imagen no incluía los cambios.
- **Instalación en disco.** El instalador fallaba con «no hay ningún disco de destino disponible»; se corrigió la detección de medio live, pero **no se ha vuelto a probar.**

4.3 No empezado

- Voz (STT/TTS) integrada en la interfaz.
- Autoadministración real del sistema por parte del agente.
- Base de conocimiento de TI.
- Entrenamiento del LLM propio.
- Aplicaciones gráficas más allá del navegador.

5. BLOQUEANTE 1 — Cerrar el arranque automático

Objetivo: encender el portátil y llegar a la interfaz Her sin escribir nada (salvo la API key la primera vez).

Ficheros implicados:

- `build/configs/agentos_defconfig` — `getty` en `tty2`, `seatd`
- `build/overlays/etc/init.d/S49agentos-firstboot` — asistente en `tty1`
- `build/overlays/etc/init.d/S71weston` — compositor Wayland
- `build/overlays/etc/init.d/S99agentos` — AgentD + interfaz
- `build/overlays/etc/xdg/weston/weston.ini` — kiosk
- `board/agentos/post-build.sh` — autologin por `inittab`
- `board/agentos/grub.cfg` y `grub-iso.cfg` — consola en `tty1`

Orden de arranque previsto:

<code>S01install</code>	→ mensaje de bienvenida (solo en live)
<code>S49agentos-firstboot</code>	→ asistente en <code>tty1</code> (solo si no hay <code>/etc/agentos/.configured</code>)
<code>S60agentos-pip</code>	→ instala faster-whisper y sounddevice en segundo plano
<code>S70seatd</code>	→ gestor de asientos (lo instala Buildroot)
<code>S71weston</code>	→ compositor Wayland en kiosk
<code>S99agentos</code>	→ AgentD + interfaz gráfica (cog)

Pasos:

1. Recompilar con el comando de la sección 2.
2. Arrancar la imagen en VirtualBox (o QEMU) y observar.
3. Si Weston no arranca, revisar `/var/log/weston.log` y confirmar que `seatd` está corriendo (`ps | grep seatd`).
4. Si la interfaz no aparece, revisar `/var/log/agentos-gui.log` y `/var/log/agentd.log`.
5. Consola de respaldo: `Ctrl+Alt+F2` (autologin como `agent`).

Criterio de aceptación: encender → (asistente la primera vez) → aparece el orbe de la interfaz Her a pantalla completa, sin login.

Riesgo conocido: en VirtualBox el driver de vídeo debe ser `VMSVGA` y el kernel necesita `CON-FIG_DRM_VMWGFX` (ya añadido). Si se usa `VBoxVGA` no habrá aceleración.

6. BLOQUEANTE 2 — Instalación en disco

Objetivo: que `agentos-install` funcione en el portátil real.

Contexto del fallo: el instalador marcaba `/dev/sda` como «medio de instalación» porque usaba la firma de disco (`a6e705a1`) para detectar el medio live. Cualquier disco con una instalación previa de AgentOS quedaba excluido. **Ya corregido:** ahora solo se excluye el disco del que se arranca (dispositivo raíz) y los medios ISO9660.

Pasos:

1. Recompilar y arrancar el ISO.
2. Ejecutar `sudo agentes-install`.
3. Verificar que el disco de destino aparece en la lista.
4. Instalar, retirar el medio y reiniciar.

Criterio de aceptación: el sistema arranca desde el disco interno, sin el medio de instalación, y llega a la interfaz.

Nota para pruebas en VirtualBox: hacen falta **dos discos** (el ISO como unidad óptica y un disco duro vacío como destino). Si el disco duro ya tiene la imagen `.img` escrita, el instalador lo tratará correctamente como destino, pero conviene usar uno limpio.

7. Voz (STT + TTS)

Objetivo: hablarle al sistema y que responda hablando, sin tocar el teclado.

Ficheros existentes (esbozos): `ui/voice/stt.py`, `ui/voice/tts.py`.

Dependencias: `faster-whisper` y `sounddevice` se instalan con pip en el primer arranque con red (`S60agentos-pip`). `portaudio` ya está en el `defconfig`.

Pasos:

1. Implementar STT con `faster-whisper` usando un modelo pequeño (`tiny` o `base`) para que quepa en 6-7 GB de RAM. El modelo se descarga la primera vez.
2. Implementar TTS. Opciones por orden de preferencia: `piper` (ligero, offline, buena calidad), `es-peak-ng` (muy ligero, calidad baja), o un servicio en la nube si el usuario acepta latencia.
3. Añadir detección de actividad de voz (VAD) para no enviar silencio al STT.
4. Integrar en la interfaz: botón/orbe que se activa al hablar, y reproducción de la respuesta.
5. Añadir un modo «conversación continua» opcional.

Criterio de aceptación: el usuario dice «hola» al micrófono y el orbe responde en voz alta en menos de 3 segundos.

Riesgo: el audio en el portátil objetivo puede requerir ajuste de ALSA (`alsamixer`). Verificar con `aplay` y `arecord` antes de integrar.

8. Autoadministración del sistema

Objetivo: que el agente pueda ejecutar acciones reales sobre el sistema operativo, no solo conversar.

Ficheros: `core/agent/tools/base.py` (infraestructura de herramientas), `core/agent/agent.py`.

Herramientas mínimas a implementar:

Herramienta	Descripción	Riesgo
<code>bash_exec</code>	Ejecutar comandos de shell	Alto — requiere confirmación
<code>read_file</code> / <code>write_file</code>	Leer y escribir ficheros	Medio
<code>list_dir</code>	Listar directorios	Bajo
<code>system_info</code>	CPU, RAM, disco, red, temperatura	Bajo
<code>service_control</code>	Arrancar/parar servicios	Medio
<code>package_install</code>	Instalar paquetes	Alto
<code>network_config</code>	Ver y cambiar configuración de red	Alto

Reglas de seguridad obligatorias:

1. Las herramientas de riesgo alto **deben pedir confirmación explícita** al usuario antes de ejecutarse, mostrando el comando exacto.
2. Lista negra de patrones destructivos (`rm -rf /` , `mkfs` , `dd of=/dev/` , `wipefs`) que nunca se ejecutan sin confirmación escrita.
3. Registrar todas las acciones en un log auditable (`/var/log/agentos-actions.log`).
4. El agente debe poder explicar qué va a hacer **antes** de hacerlo.

Criterio de aceptación: el usuario dice «instala htop y dime cuánta RAM libre queda» y el agente pide confirmación, ejecuta y responde con el dato real.

9. Conocimiento de TI (base de conocimiento local)

Objetivo: que el agente tenga «todo el conocimiento de TI posible» disponible sin depender de internet.

Enfoque recomendado: RAG (generación aumentada por recuperación) sobre documentación local.

Pasos:

1. Recopilar documentación: `man` pages del sistema, `--help` de las herramientas instaladas, documentación de Buildroot, guías de administración de Linux.
2. Trocear y generar embeddings. Opciones: `sentence-transformers` (local, ~100 MB) o un modelo pequeño de embeddings vía Ollama.
3. Almacenar en una base vectorial ligera. Opciones: `sqlite-vec` (encaja con el SQLite que ya trae Python), `chromadb` o `faiss`.
4. Añadir una herramienta `search_docs` al agente que consulte la base antes de responder.
5. Indexar también el propio código de AgentOS para que el agente se entienda a sí mismo.

Criterio de aceptación: el usuario pregunta «cómo cambio la contraseña de un usuario» y el agente responde con el comando correcto citando la fuente local.

Nota: los scripts `training/data/collect_docs.py` y `collect_manpages.py` ya existen como punto de partida.

10. LLM propio

Objetivo: «su LLM también lo quiero crear desde 0».

Aclaración importante que hay que dar al usuario: entrenar un LLM desde cero (preentrenamiento) requiere miles de GPU-hora y un corpus de billones de tokens. **No es viable** en un portátil de 8 GB. Lo que sí es viable y da un resultado equivalente para el objetivo es:

- **Opción A (recomendada):** fine-tuning por LoRA sobre un modelo base abierto (Llama 3.1 8B, Qwen 2.5 7B, Mistral 7B). Se entrena un adaptador pequeño (~50 MB) que especializa el modelo en el dominio de AgentOS y administración de sistemas.
- **Opción B:** destilación — usar un modelo grande (DeepSeek v4.1-flash vía Ollama Cloud) para generar un dataset de instrucciones de TI, y entrenar con él un modelo pequeño.
- **Opción C:** entrenar un modelo muy pequeño desde cero (tipo nanoGPT, 10-50 M de parámetros) solo con fines demostrativos. No será útil en producción.

Ficheros existentes: `training/finetune/train.py`, `training/finetune/lora_config.yaml`, `training/data/format_dataset.py`.

Pasos (Opción A):

1. Generar el dataset: usar el modelo grande para crear pares instrucción/respuesta sobre administración de Linux, AgentOS y las herramientas del agente.
2. Formatear con `format_dataset.py` al formato de chat del modelo base.
3. Entrenar el LoRA. **No en el portátil:** usar la VPS (16 vCPU) o un servicio con GPU. Con CPU pura, un LoRA de 8B tarda días.
4. Convertir el adaptador a GGUF y fusionarlo con el modelo base.
5. Colocar el GGUF en `/data/models/` y apuntar `AGENTOS_MODEL_PATH` a él.
6. Comparar respuestas antes/después con un conjunto de evaluación fijo.

Criterio de aceptación: el modelo ajustado responde correctamente a preguntas sobre AgentOS y administración de Linux, y supera al modelo base en el conjunto de evaluación.

11. Aplicaciones gráficas

Objetivo: «cuando sea necesario mostrar información al usuario... editores, película, ssh, navegador».

Estado: el compositor Weston y el navegador `cog` (WPE WebKit) ya están en el defconfig. `cog` es lo que muestra la interfaz Her.

Pasos:

1. **Navegador:** ya cubierto por `cog`. Añadir una herramienta al agente para abrir una URL en una ventana nueva.
2. **Editor de texto:** opciones ligeras en Buildroot: `nano`, `vim`, `mousepad` (GTK, pesado). Recomendado: `nano` en terminal y, si se quiere gráfico, evaluar `leafpad` o `featherpad`.
3. **SSH:** `openssh` ya está en el defconfig. Añadir una herramienta `ssh_connect` que abra una sesión en una terminal gráfica (`foot` o `weston-terminal`).
4. **Vídeo:** `mpv` o `gst-play`. Requiere códecs; evaluar el impacto en el tamaño de la imagen.
5. **Terminal gráfica:** `foot` (Wayland nativo, muy ligero) o `weston-terminal`.
6. **Lanzador genérico:** una herramienta `launch_app` que resuelva ficheros `.desktop` y arranque la aplicación bajo Weston.

Criterio de aceptación: el usuario dice «ábreme el navegador en wikipedia.org» y aparece una ventana con la página, sobre la interfaz Her.

Riesgo: cada aplicación añade tamaño a la imagen. Vigilar que el ISO no supere lo razonable para un USB de 2 GB.

12. Instalación en el portátil real y pulido final

Pasos:

1. Grabar `agentos.img` en un USB: `sudo dd if=agentos.img of=/dev/sdX bs=4M status=progress conv=fsync`.
2. Arrancar el portátil desde el USB (BIOS o UEFI).
3. Verificar que la interfaz aparece y que la voz funciona.
4. Ejecutar `sudo agentes-install` e instalar en el disco interno.
5. Reiniciar sin el USB y comprobar el arranque completo.
6. **Cambiar las contraseñas por defecto** (`passwd` para `agent` y `root`).
7. Medir: tiempo de arranque, RAM libre con el modelo cargado, temperatura.

Criterio de aceptación: el portátil arranca solo, muestra la interfaz, responde por voz y por texto, y ejecuta acciones reales sobre el sistema.

13. Trampas conocidas (errores ya resueltos — no repetir)

1. El Makefile pierde los tabuladores.

Los ficheros `Makefile` necesitan **tabuladores reales** en las recetas, no espacios. Editores y herrami-

entas que normalizan espacios rompen el build con `missing separator`. Solución: escribir el fichero con Python usando `\t` explícitos y verificar con `grep -P '^\\t' Makefile`. **Nunca** uses `file_write` ni `file_str_replace` para el Makefile.

2. GCC 13/14 de Ubuntu 24.04 rompe `host-m4`.

Buildroot 2024.11.1 falla al compilar `host-m4-1.4.19` con GCC moderno. Solución: `sudo apt install gcc-12 g++-12` y compilar con `HOSTCC=gcc-12 HOSTCXX=g++-12`.

3. CMake 3.28+ rechaza `jpeg-turbo 2.1.5`.

Error: `Compatibility with CMake < 3.5 has been removed`. Solución: en `external.mk`, añadir `JPEG_TURBO_CONF_OPTS += -DCMAKE_POLICY_VERSION_MINIMUM=3.5`. Si otro paquete falla igual, aplicar el mismo patrón y borrar su `.stamp_configured`.

4. BusyBox `start-stop-daemon` no soporta `-d`.

Imprime su ayuda y aborta. Solución: hacer el `cd` dentro del comando shell: `-x /bin/sh -- -c "cd $DIR && exec ..."`.

5. `BR2_TARGET_GENERIC_GETTY_AUTOLOGIN` no existe.

No está en Buildroot 2024.11.1; se ignora en silencio. Solución: parchear `/etc/inittab` en `post-build.sh` con `tty2::respawn:/bin/login -f agent`.

6. Orden de arranque de `seatd`.

Buildroot instala `seatd` como `S70seatd`. Weston **debe** arrancar después (`S71weston`), o falla al no encontrar el gestor de asientos.

7. `fail to initialize ptp_kvm` es inofensivo.

El kernel intenta sincronizar el reloj con KVM; en VirtualBox no aplica. Se puede ignorar.

8. El repositorio local puede estar desincronizado.

Parte del trabajo se ha subido directamente por la API de GitHub. **Haz siempre `git pull` antes de editar** y verifica el contenido real del fichero antes de parchearlo.

9. Buildroot es incremental.

Para reintentar un paquete concreto, borra sus stamps:

```
rm -f build/output/build/<paquete>-<version>/stamp_configured \
    build/output/build/<paquete>-<version>/stamp_built \
    build/output/build/<paquete>-<version>/stamp_installed
```

No hace falta recompilar todo.

14. Anexo — comandos de referencia

```
# Compilar la imagen completa
cd ~/newos && git pull
HOSTCC=gcc-12 HOSTCXX=g++-12 make build J=16

# Probar en QEMU (BIOS)
make run-qemu-iso

# Probar en QEMU (UEFI)
qemu-system-x86_64 -enable-kvm -cpu host -m 4G -smp 4 \
  -cdrom build/output/images/agentos.iso \
  -bios /usr/share/ovmf/OVMF.fd -device virtio-vga

# Grabar en USB
sudo dd if=build/output/images/agentos.img of=/dev/sdX bs=4M status=progress conv=fsync
c

# Dentro del sistema: ver logs
cat /var/log/agentd.log
cat /var/log/agentos-gui.log
cat /var/log/weston.log

# Dentro del sistema: reconfigurar el LLM
agentos-config

# Dentro del sistema: reiniciar servicios
/etc/init.d/S99agentos restart
/etc/init.d/S71weston restart
```

15. Resumen de prioridades

1. **Verificar el arranque automático** (sección 5) — es el bloqueante actual.
2. **Verificar la instalación en disco** (sección 6).
3. **Voz** (sección 7) — es lo que más cambia la experiencia.
4. **Autoadministración** (sección 8) — es el corazón del «agente que se administra a sí mismo».
5. **Conocimiento de TI** (sección 9).
6. **LLM propio** (sección 10) — requiere GPU; planificar aparte.
7. **Aplicaciones gráficas** (sección 11).
8. **Instalación en el portátil real** (sección 12).